CSE3144

ADVANCED DATA STRUCTURES ASSIGNMENT

Implementing Advanced Data Structures to Solve Real-World Scenarios

NAME

Chinmayee Khanna

ROLL NUMBER

2427030720

1

PART 1

The Genome Archive

Huffman coding

2

PART 2

Station-Name Suggestions

Trie

3

PART 3

Rover Firmware Queue

Pairing heap

Part 1: Scenario 14 - The Genome Archive

- DNA sequences consist of the bases A, C, G and T
- Different bases may occur at different frequencies
- Objective: reduce storage without losing any information
- The program compares an AT-rich sequence with a near-uniform one

SELECTED TECHNIQUE

Huffman coding

Why frequency-based codes help?

Fixed length: every base costs 2 bits

A 00	C 01	G 10	T 11

Frequency-based

A 0	T 10	C 110	G 111

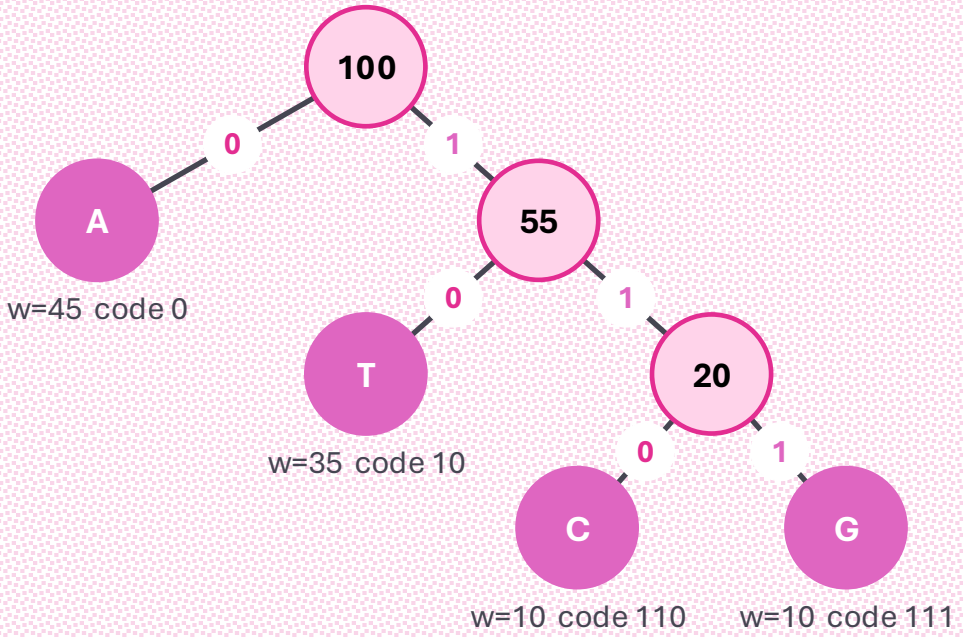
Frequent bases get short codes; rare bases get longer ones.

Suppose A 45%, T 35%, C 10%, G 10% gives 0.45·1 + 0.35·2 + 0.10·3 + 0.10·3 = 1.75 bits per base (not program output).

Algorithm and Justification

Huffman coding algorithm

1. Count the frequency of each DNA base
2. Insert the symbols into a minimum-priority queue
3. Repeatedly combine the two least-frequent nodes
4. Construct the Huffman tree
5. Assign binary codes to the leaves (left 0, right 1)
6. Encode and decode to verify losslessness



Why Huffman? gives an optimal prefix code for the observed frequencies; lossless, with short codes for common bases.

Rejected: fixed-length coding. Always 2 bits per base, so it cannot exploit uneven symbol frequencies.

Experimental Results and Observations

Metric	AT-rich DNA	Uniform DNA
Sequence length	100,000 bases	100,000 bases
Encoded length	179,790 bits	200,000 bits
Average bits per base	1.7979	2.0000
Round-trip verification	**SUCCESS**	**SUCCESS**
Saving versus two-bit baseline	Approximately 10.10%	0%

- **AT-rich:** some bases occur more often, so Huffman coding compresses the sequence
- **Uniform:** close to the two-bit baseline, so no reduction versus fixed-length encoding

The experimental results support the choice of Huffman coding for non-uniform DNA data. In the AT-rich experiment, A and T together account for approximately 80% of the sequence, so the Huffman tree assigns shorter codes to the frequent bases. The measured average is 1.798510 bits per base compared with the 2.00-bit fixed-length baseline, giving approximately 10.07% savings. In the uniform experiment, all four bases have nearly equal frequencies, and the Huffman tree assigns two bits to every symbol.

Part 2: Scenario 11- Station-Name Suggestions

- Store railway station names
- Search for exact station names
- Suggest stations matching a typed prefix
- Support deletion and reinsertion

SELECTED DATA STRUCTURE

Trie (prefix tree)

Example station names

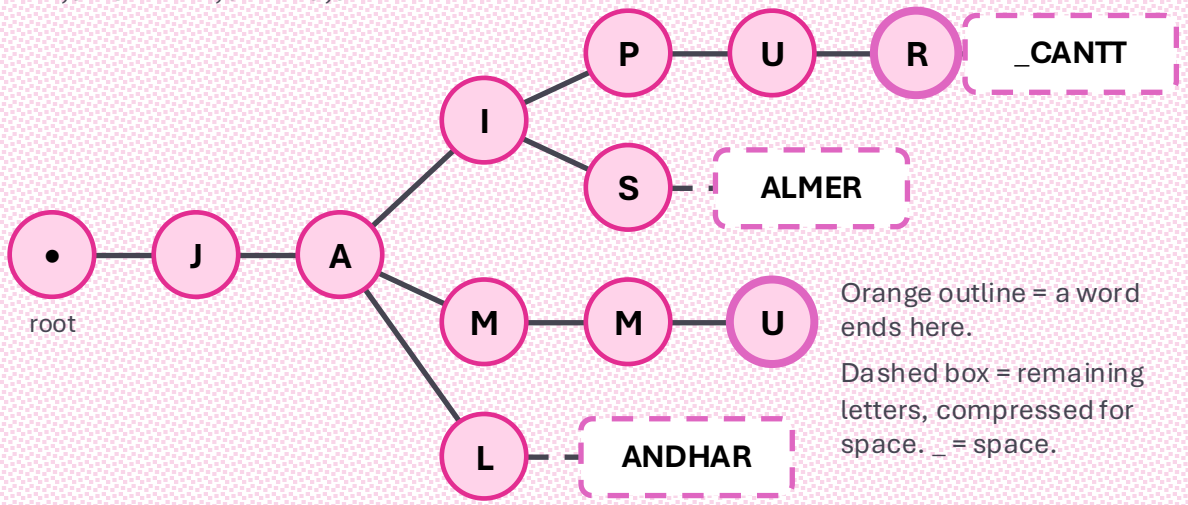
JAIPUR
JAIPUR CANTT
JAISALMER
JAMMU
JALANDHAR
JABALPUR
JAGDALPUR

Shared prefix = shared path. Every name above starts with J → A, and JAIPUR and JAIPUR CANTT share all six letters of JAIPUR; each shared letter is stored once.

Trie Structure and Justification

- **Insert:** walk the path, creating a node for each missing character; mark the last node as end-of-word
- **Exact search:** follow the characters; success only if the final node is marked end-of-word
- **Prefix lookup:** follow P characters to reach the prefix node
- **List words:** depth-first walk below the prefix node, children in alphabetical order
- **Delete:** unmark end-of-word and prune only childless, unmarked nodes, so shared paths survive

Prefix trie (subset): JAIPUR, JAIPUR CANTT, JAISALMER, JAMMU, JALANDHAR



Insert	Exact search	Find prefix node
O(L)	O(L)	O(P)

Rejected: hash table. Supports exact lookup, but not efficient prefix suggestions without scanning every key.

L = station-name length, P = prefix length. Listing the suggestions costs extra, proportional to the size of the subtree visited.

Results and Observations

Test	Result
Search JAIPUR	**FOUND**
Search JAIPUR CANTT	**FOUND**
Search DELHI	**NOT FOUND**
Prefix JA	Multiple suggestions
Prefix JAI	JAIPUR, JAIPUR CANTT, JAISALMER
Prefix JAL	JALANDHAR
Prefix X	No suggestions
Delete JAGDALPUR	**Successful**
Reinsert JAGDALPUR	**Successful**

Prefix JA, alphabetical

JABALPUR

JAGDALPUR

JAIPUR

JAIPUR CANTT

JAISALMER

Observation: The Trie supports exact search, prefix suggestions, deletion and reinsertion.

Part 3: Scenario 10- Rover Firmware Queue

- Implement a **mergeable priority queue**
- Operations: **insert, meld, find-min, decrease-key, extract-min**
- Count the **links** performed during extract-min
- Test against a **sorted reference** using **100,000** randomized operations

SELECTED DATA STRUCTURE

Pairing heap

Strict node restriction

Each node may contain only:

- **Key**
- **First-child pointer**
- **Next-sibling pointer**

Not allowed:

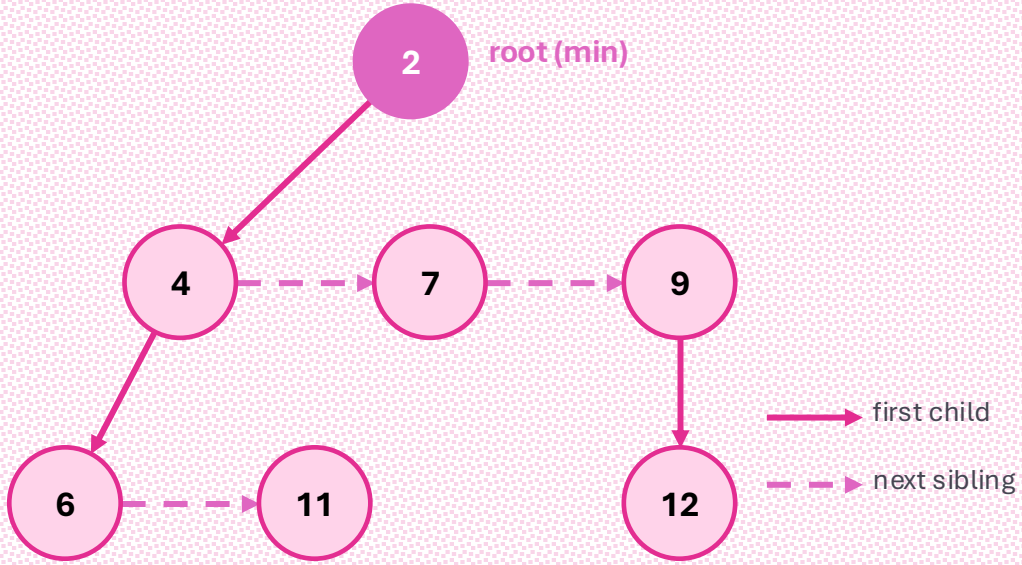
- ✕ **degree field**
- ✕ **mark bit**
- ✕ **parent pointer**

The node stays minimal: three fields per node.

Pairing Heap and Justification

- **Min key at the root** of a heap-ordered tree
- **Child-sibling pointers:** first child and next sibling
- **Insert:** meld a singleton node with the heap
- **Meld:** the root with the larger key becomes first child of the other
- **Extract-min:** remove the root, then pair and combine its child subtrees

Conceptual pairing heap (min-heap)



Rejected: binary heap. Less convenient for meld and for updating the priority of an arbitrary node.

Complexity caveat: with no parent pointers, decrease-key must search for the node's parent: O(n) worst case here, not O(1).

Operations and Complexity

Operation	Description	Standard analysis	This implementation
Insert	Add a new priority-queue item	O(1)	O(1): one link
Meld	Combine two heaps	O(1)	O(1): one link
Find-min	Read the root key	O(1)	O(1): read root
Decrease-key	Reduce an item's key and restore heap order	Amortized; not claimed O(1)	**O(n) worst case (parent search)**
Extract-min	Remove the minimum and consolidate its child subtrees	O(log n) amortized	Links counted per call

Key trade-off: the no-parent-pointer restriction keeps nodes minimal but moves cost into decrease-key. Standard amortized analysis assumes cheap access to the node's parent.

Experimental Results

Metric	Value
Randomized operations	100,000 (per assignment specification)
Total extract-min links	288,055
Reference comparison failures	0
Successful inserts	39,807
Successful Decrease-Key operations	20,032
Validation result	SUCCESS

Observation

Randomized testing compares the priority queue against a reference implementation to check correctness across operations.

Comparison of the Three Approaches

Scenario	Technique	Main Advantage	Main Trade-off
Genome Archive	**Huffman coding**	Lossless compression	Benefit depends on symbol frequencies
Station Suggestions	**Trie**	Natural prefix search	Memory required for character nodes
Rover Queue	**Pairing heap**	Mergeable priority queue	Parent search makes decrease-key O(n) worst case in this implementation

Key lesson: data-structure selection depends on the operations, constraints and performance goals of the problem.

Conclusion

1. Huffman coding reduces storage when symbol frequencies are uneven.
2. Tries naturally support exact and prefix-based string searches.
3. Pairing heaps support priority-queue operations and melding under the required node restriction.
4. Testing validates implementation behavior.
5. Appropriate data-structure selection improves efficiency.